

torch.linalg.eigh: 80–120× Performance Cliff at $n = 33$ and How to Fix It

Sijing Du

February 2026

Abstract

`torch.linalg.eigh` suffers a catastrophic 80–120× performance cliff when the matrix dimension crosses $n = 32 \rightarrow 33$ for batched inputs on GPU. The root cause is PyTorch’s size-dependent dispatch logic, which gates the genuinely batched cuSOLVER API (`cusolverDnXsyevBatched`) behind $n \leq 32$. We demonstrate the problem, evaluate the MAGMA workaround, and show that calling `cusolverDnXsyevBatched` unconditionally—matching JAX’s fix (PR #31375)—eliminates the cliff entirely with 32–77× speedup and no numerical regression. The fix requires changing a single dispatch condition in PyTorch’s C++ source.

Test environment. All benchmarks: NVIDIA GeForce RTX 4080, CUDA 12.8, PyTorch 2.10.0+cu128, float64, Ubuntu 22.04.5 LTS (WSL2). Timing via CUDA events (3 warmup, 5 timed repetitions).

1 Observation in stock PyTorch eigh performance

`torch.linalg.eigh` applied to a batch of (B, n, n) symmetric matrices exhibits smooth scaling for $n \leq 32$, then an abrupt 80–140× slowdown at $n = 33$.

Table 1: Stock `torch.linalg.eigh` timing (ms) vs. matrix size n .

n	$B = 64$		$B = 1024$	
	time (ms)	ratio	time (ms)	ratio
8	0.29		0.91	
16	0.52		2.86	
24	0.85		5.49	
28	0.93		7.22	
32	1.05	1.0×	8.30	1.0×
33	77.20	73×	1137	137×
34	78.87	75×	1153	139×
40	87.37	83×	1296	156×
48	97.31	93×	1491	180×
64	122.1	116×	1874	226×
96	246.2	234×	3685	444×
128	316.0	301×	4783	576×

1.1 Root cause: PyTorch’s dispatch logic

The dispatch lives in `aten/src/ATen/native/cuda/linalg/BatchLinearAlgebraLib.cpp`, function `linalg_eigh_cusolver` (line ~1614):

```
if (use_cusolver_syevj_batched_
    && batchCount(eigenvectors) > 1
    && eigenvectors.size(-1) <= 32) {
```

```

    // genuinely batched: calls cusolverDnXsyevBatched
    linalg_eigh_cusolver_syevj_batched();
} else if (scalar_type == kFloat
    && size >= 32 && size <= 512) {
    // single-matrix Jacobi, LOOPED over batch
    linalg_eigh_cusolver_syevj();
} else {
    // single-matrix divide-and-conquer, LOOPED
    linalg_eigh_cusolver_syevd();
}

```

The branching logic:

- $n \leq 32$, **batch** > 1: correctly routes to `linalg_eigh_cusolver_syevj_batched()`, which (since PR #155695, cuSOLVER ≥ 11701) calls `cusolverDnXsyevBatched`—a genuinely batched API that processes all B matrices in a single call. This is fast.
- $n > 32$, **batch** > 1: falls through to the `else if` or `else` branches, which call single-matrix wrappers (`syevj` or `syevd`) in a *loop*. Each matrix is solved sequentially with a separate kernel launch. This is where the 80–120 $\times$ cliff comes from.

The $n \leq 32$ gate is a leftover from the old `syevjBatched` Jacobi API, which was replaced by the size-unlimited `cusolverDnXsyevBatched` in PR #155695 (June 2025). The top-level dispatch condition was never updated.

2 High-level MAGMA workaround (no C++ level change)

PyTorch exposes MAGMA as an alternative backend:

```
torch.backends.cuda.preferred_linalg_library('magma')
```

MAGMA’s batched eigensolver has no $n = 32$ cliff and scales smoothly. However, it is significantly slower than cuSOLVER’s fused kernel for $n \leq 32$:

Table 2: `torch.linalg.eigh` performance: cuSOLVER (default) vs. MAGMA backend ($B = 64$, `float64`).

n	default (ms)	MAGMA (ms)	MAGMA speedup
32	0.90	4.24	0.2 $\times$ (slower)
33	69.88	4.44	15.7 $\times$
48	93.12	7.76	12.0 $\times$
64	116.7	10.01	11.7 $\times$

MAGMA is a valid workaround for $n > 32$ (7–16 $\times$ faster than default), but it comes with two drawbacks:

1. **Requires size-aware dispatch in user code:** `preferred_linalg_library` is process-wide, but it can be toggled locally with `save/restore` around each call (we implemented this in `size_aware_eigh` in our codebase). Still, this pushes dispatch complexity onto the user rather than fixing it in PyTorch, and MAGMA is $\sim 4\times$ slower than the cuSOLVER batched kernel for $n \leq 32$.
2. **Suboptimal for $n > 32$:** as we show in Section 3, `cusolverDnXsyevBatched` is 2–4 $\times$ faster than MAGMA for the same sizes.

3 Ultimate solution: dispatch updated cuSOLVER batched eigh (`cusolverDnXsyevBatched`), as in JAX PR #31375

3.1 The fix

`cusolverDnXsyevBatched` (available since cuSOLVER 11.7.1 / CUDA 12.6) is a genuinely batched divide-and-conquer eigensolver with **no matrix-size limitation**. PyTorch already wraps it and uses it for $n \leq 32$. The fix is to remove the $n \leq 32$ gate:

```

// Proposed change in linalg_eigh_cusolver:
#ifdef USE_CUSOLVER_64_BIT_XSYEV_BATCHED
// cusolverDnXsyevBatched works for all n.
// See jax-ml/jax#31375 for precedent.
if (batchCount(eigenvectors) > 1) {
    linalg_eigh_cusolver_syeval_batched(...);
} else if (scalar_type == kFloat
           && size >= 32 && size <= 512) {
    // Non-batched: preserve original dispatch.
    linalg_eigh_cusolver_syeval(...);
} else {
    linalg_eigh_cusolver_syevd(...);
}
#else
// Fallback for cuSOLVER < 11.7.1:
// keep original n<=32 gate.
...
#endif

```

This is similar to the approach JAX took in [jax-ml/jax#31375](#) (merged September 2025): route all batched `eigh` through `cusolverDnXsyevBatched`, replacing the per-matrix `syevd` loop.

3.2 Verification via C++ extension

To verify without rebuilding PyTorch, we wrote a minimal C++ extension (`my_eigh.cpp`) that calls `cusolverDnXsyevBatched` directly for all matrix sizes, loaded via `torch.utils.cpp_extension.load`.

Correctness. Eigenvalue max $|\Delta| = 0$; eigenvector residual $\|Av - \lambda v\| = 2.1 \times 10^{-13}$ (same order as stock PyTorch’s 1.7×10^{-13}). **PASS.**

Performance. Table 3 and Table 4 show the results.

Table 3: Stock `eigh` vs. `XsyevBatched` extension ($B = 64$, `float64`).

n	stock (ms)	XsyevBatched (ms)	speedup
8	0.29	0.20	1.5×
16	0.52	0.41	1.3×
24	0.85	0.74	1.1×
28	0.93	0.78	1.2×
32	1.05	0.88	1.2×
33	77.20	1.68	46×
34	78.87	1.43	55×
40	87.37	1.53	57×
48	97.31	1.77	55×
64	122.1	2.34	52×
96	246.2	6.74	37×
128	316.0	9.88	32×

3.3 Comparison: all three methods

Table 5 summarizes the three dispatch strategies for the representative case $B = 64$:

`XsyevBatched` is the best of both worlds: fast for $n \leq 32$ (matching current `cuSOLVER`) and fast for $n > 32$ (2–4× faster than `MAGMA`).

3.4 Summary

1. The $n = 32 \rightarrow 33$ performance cliff is **completely eliminated**—timing scales smoothly across all n .
2. **32–77×** speedup for $n > 32$, with no regression for $n \leq 32$.

Table 4: Stock `eigh` vs. `XsyevBatched` extension ($B = 1024$, `float64`).

n	stock (ms)	XsyevBatched (ms)	speedup
8	0.91	0.75	1.2×
16	2.86	2.66	1.1×
24	5.49	5.47	1.0×
28	7.22	6.92	1.0×
32	8.30	7.81	1.1×
33	1137	15.06	75×
34	1153	15.01	77×
40	1296	17.36	75×
48	1491	21.89	68×
64	1874	30.44	62×
96	3685	96.10	38×
128	4783	145.7	33×

Table 5: Three-way comparison ($B = 64$, `float64`). Best time per row in **bold**.

n	default (ms)	MAGMA (ms)	XsyevBatched (ms)
32	0.90	4.24	0.88
33	69.88	4.44	1.68
48	93.12	7.76	1.77
64	116.7	10.01	2.34

3. **Numerically identical** to stock PyTorch (same cuSOLVER API, just called unconditionally).
4. The fix is a **one-line dispatch change**, guarded by an existing `#ifdef`, using an API PyTorch already wraps.
5. This matches what [JAX shipped in September 2025](#).

A Reproducer

```
import torch

device = "cuda"
B = 1024
dtype = torch.float64

for n in [31, 32, 33, 34, 48, 64]:
    x = torch.randn(B, n, n, device=device, dtype=dtype)
    a = (x + x.mT) / 2

    # warmup
    for _ in range(5):
        torch.linalg.eigh(a)
    torch.cuda.synchronize()

    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)
    start.record()
    for _ in range(10):
        torch.linalg.eigh(a)
    end.record()
    torch.cuda.synchronize()
    t = start.elapsed_time(end) / 10
    print(f"eigh n={n}: {t:.2f} ms")
```

B C++ extension source (my_eigh.cpp)

```
#include <torch/extension.h>
#include <c10/cuda/CUDASTream.h>
#include <cusolverDn.h>

static cusolverDnHandle_t get_handle() {
    static thread_local cusolverDnHandle_t h = nullptr;
    if (!h) cusolverDnCreate(&h);
    cusolverDnSetStream(h,
        c10::cuda::getCurrentCUDASTream().stream());
    return h;
}

std::tuple<torch::Tensor, torch::Tensor>
custom_batched_eigh(torch::Tensor input, bool upper) {
    TORCH_CHECK(input.is_cuda(),
        "input must be a CUDA tensor");
    TORCH_CHECK(input.dim() >= 2
        && input.size(-1) == input.size(-2),
        "input must be square");

    const auto dtype = input.scalar_type();
    const int64_t n = input.size(-1);
    const int64_t batch = input.numel() / (n * n);

    // Clone: cuSOLVER overwrites input with eigenvectors.
    // Symmetric => row-major == column-major, no transpose needed.
    auto vectors = input.contiguous().clone();
    auto values = torch::empty({batch, n}, input.options());
    auto info = torch::zeros({batch},
        input.options().dtype(torch::kInt32));

    auto handle = get_handle();
    cusolverDnParams_t params;
    cusolverDnCreateParams(&params);

    auto jobz = CUSOLVER_EIG_MODE_VECTOR;
    auto uplo = upper ? CUBLAS_FILL_MODE_UPPER
        : CUBLAS_FILL_MODE_LOWER;
    auto cuda_dtype = (dtype == torch::kFloat64)
        ? CUDA_R_64F : CUDA_R_32F;

    // Query workspace, allocate, compute
    size_t work_device_sz = 0, work_host_sz = 0;
    cusolverDnXsyevBatched_bufferSize(
        handle, params, jobz, uplo, n, cuda_dtype,
        vectors.data_ptr(), n, cuda_dtype,
        values.data_ptr(), cuda_dtype,
        &work_device_sz, &work_host_sz, batch);

    auto work_device = torch::empty(
        {static_cast<int64_t>(work_device_sz)},
        input.options().dtype(torch::kUInt8));
    std::vector<uint8_t> work_host(work_host_sz);

    cusolverDnXsyevBatched(
        handle, params, jobz, uplo, n, cuda_dtype,
        vectors.data_ptr(), n, cuda_dtype,
        values.data_ptr(), cuda_dtype,
        work_device.data_ptr(), work_device_sz,
        work_host.data(), work_host_sz,
        info.data_ptr<int>(), batch);

    cusolverDnDestroyParams(params);

    // cuSOLVER writes column-major eigenvectors into our
    // row-major buffer; transpose back for PyTorch.
    vectors = vectors.view({batch, n, n})
        .mT().contiguous();
    auto out_shape = input.sizes().vec();
    vectors = vectors.reshape(out_shape);
    auto val_shape = std::vector<int64_t>{
        out_shape.begin(), out_shape.end() - 1);
    values = values.reshape(val_shape);

    return {values, vectors};
}

PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) {
    m.def("eigh", &custom_batched_eigh,
```

```

        "Batched eig via cusolverDnXsyevBatched",
        py::arg("input"), py::arg("upper") = false);
}

```

Load and benchmark:

```

from torch.utils.cpp_extension import load
ext = load("my_eigh", sources=["my_eigh.cpp"],
          extra_cflags=["-O3"],
          extra_include_paths=[f"{cuda_home}/include"],
          extra_ldflags=[f"-L{cuda_home}/lib64",
                        "-lcusolver", "-lcudart"])
vals, vecs = ext.eigh(A)  # (B,n,n) -> (B,n), (B,n,n)

```